

Motion Planning for a Barista Mobile Manipulator in a Dynamic Restaurant Environment

Francesco Marani (6460216), Javier Gil (6563953), Núria Seguí (6299946), Simone Utili (6568688)

Abstract—This project investigates motion planning for an autonomous mobile manipulator operating in a restaurant environment. The robot, composed of a 3-DOF differential-drive base and a 7-DOF robotic arm, is tasked with transporting a drink from a bar to a table while avoiding static furniture and dynamic human obstacles. A hierarchical planning framework is evaluated, combining a global sampling-based planner (PRM, RRT or RRT*) with a local Model Predictive Controller (MPC) to ensure smooth trajectories under kinematic and safety constraints. The approach is validated in a PyBullet simulation and compared against a standalone MPC baseline. Quantitative results demonstrate that integrating a global planner significantly improves success rate and robustness to local minima, particularly in cluttered and dynamic environments.

I. INTRODUCTION

Autonomous navigation in human-populated environments is a major challenge in robotics, especially for service robots in confined spaces. Mobile manipulators further increase this complexity due to their high-dimensional configuration spaces and the need to coordinate base motion with manipulation while satisfying safety and stability constraints [1].

This project studies motion planning for an autonomous barista mobile manipulator in a restaurant. The Albert robot we use has a differential-drive base and a 7-DOF arm [2], tasked with picking a drink from the bar and delivering it to a table while avoiding obstacles and maintaining smooth motion.

To address this study, we follow a three-step process. Firstly, we test Model Predictive Control (MPC) alone, as a baseline to identify the limits of using only a local planner. Secondly, we combine it with sampling-based global planners, namely Probabilistic Roadmap (PRM), Rapidly-exploring Random Tree (RRT and RRT*) [3], to provide reference paths. The three methods are implemented and evaluated in a PyBullet simulation, where performance is measured in terms of success rate, execution time, smoothness, and robustness to local minima for comparison.¹

II. PROBLEM DEFINITION

This section defines the robot model, planning spaces, and constraints for the mobile manipulator navigation task in a restaurant environment.

¹The project was jointly defined by all members. Nuria developed the simulation environment, configured PRM, and handled data analysis and reporting. Francesco implemented collision avoidance in the base MPC and developed RRT/RRT*. Simone designed the base MPC, hierarchical planning framework, and visualization tools. Javi implemented the arm MPC, grasping logic, and conducted experiments and data analysis.



Fig. 1. Simulation environment: the Restaurant and the Albert robot.

A. Kinematic Modeling

The Albert mobile manipulator comprises a differential-drive Clearpath Boxer base and a 7-DOF Franka Emika Panda robotic arm. The kinematics are modeled separately for each subsystem.

Mobile Base Kinematics:

The differential-drive base has configuration $q_b = [x, y, \theta]^T \in \mathbb{R}^3$ with non-holonomic kinematics:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega, \quad (1)$$

where v and ω are linear and angular velocities, respectively, related to wheel velocities by

$$v = \frac{R}{2}(\dot{\phi}_R + \dot{\phi}_L), \quad \omega = \frac{R}{2L}(\dot{\phi}_R - \dot{\phi}_L), \quad (2)$$

with wheel radius R and track width $2L$.

Manipulator Kinematics and Dynamics:

The 7-DOF arm configuration $\mathbf{q}_a \in \mathbb{R}^7$ maps to end-effector pose via forward kinematics $\mathbf{X}_b^e = f_{k,arm}(\mathbf{q}_a)$ with Jacobian $\mathbf{J}_a(\mathbf{q}_a) \in \mathbb{R}^{6 \times 7}$ relating joint velocities to end-effector twist.

The arm dynamics follow the standard manipulator equation:

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q}_a)\ddot{\mathbf{q}}_a + \mathbf{C}(\mathbf{q}_a, \dot{\mathbf{q}}_a)\dot{\mathbf{q}}_a + \mathbf{g}(\mathbf{q}_a), \quad (3)$$

where $\boldsymbol{\tau} \in \mathbb{R}^7$ are the joint torques, $\mathbf{M}(\mathbf{q}_a) \in \mathbb{R}^{7 \times 7}$ is the inertia matrix, $\mathbf{C}(\mathbf{q}_a, \dot{\mathbf{q}}_a) \in \mathbb{R}^{7 \times 7}$ represents Coriolis and centrifugal effects, and $\mathbf{g}(\mathbf{q}_a) \in \mathbb{R}^7$ accounts for gravitational forces. This inverse dynamics model is used in the arm MPC to compute required torques from desired joint accelerations.

B. Configuration Space & Sequential Planning Strategy

The complete mobile manipulator system has a 10-DOF configuration space $\mathcal{C} = \mathbb{R}^2 \times S^1 \times (S^1)^7$. Direct whole-body planning in this high-dimensional space is computationally expensive for sampling-based methods [4]. To address this, a **sequential planning strategy** is adopted, splitting the task into four phases with reduced active degrees of freedom.



Fig. 2. The Albert robot transporting a bottle to target location.

- **Phase 1 (Approach):** The base navigates from its initial position to the bottle's location while arm remains in a resting configuration.
- **Phase 2 (Grasping):** The mobile base is fixed, and the manipulator is controlled to grasp the drink from the table.
- **Phase 3 (Transportation):** The arm is locked in a stable transport configuration to maintain the drink upright, and the mobile base navigates to the target table while avoiding obstacles.
- **Phase 4 (Delivering):** The base is stopped, and the manipulator places the drink on the target table.

The global planner (PRM, RRT, or RRT*) is applied only to the base navigation in Phase 1 and Phase 3. Phases 2 and 4 are executed using MPC directly for trajectory optimization and control.

C. Planning Spaces

Different planners work in different spaces:

- **Sampling-based planners** (PRM, RRT, RRT*) search for collision-free paths in **configuration space** ($\mathcal{C}$).
- **MPC** optimizes control in **state space**, using the current positions, velocities, and dynamics over a receding horizon.

Sampling methods plan geometrically, while MPC refines trajectories using the system dynamics.

D. Task Constraints

To ensure safe and efficient operation, the system must satisfy:

General Constraints:

- Maintain at least 0.2 m from obstacles.
- Handle dynamic obstacles moving at constant speed of 0.4 m/s.
- Stay within the 20×10 m restaurant area.

Mobile Base:

- Max linear velocity $v_{\max} = 1.5$ m/s, angular velocity $\omega_{\max} = 1.5$ rad/s.

Manipulator:

- Keep end-effector horizontal during base motion.
- Respect actuator limits: torque ($\tau \in [-87, 87]$ Nm for base joints, ± 12 Nm for wrist joints) and joint velocities ($|\dot{q}| \leq 2.61$ rad/s).
- Respect joint position limits from the URDF.

E. Evaluation Metrics

Performance of the planning strategies is assessed using the following metrics:

- **Success Rate:** Percentage of trials completed without failures due to convergence to local minima, not reaching task or hitting an obstacle.²
- **Planning Time:** Computation time required by the global planner (PRM, RRT, or RRT*) in seconds.
- **Execution Time:** Total task completion time in seconds.
- **Global Path Length:** Length of the reference path on Phase 1 produced by the global planner in meters.
- **Executed Distance:** Total distance traveled by the robot during Phase 1 in meters.
- **Trajectory Smoothness:** Measured using acceleration and jerk norms during execution.

III. MOTION PLANNING METHOD

The motion planning strategy is evaluated using both a hierarchical approach (global planner + local controller) and a standalone MPC baseline. For the mobile base, both planning strategies are tested: (1) point-to-point trajectory optimization using MPC alone [5], and (2) hierarchical planning with a global planner providing reference paths tracked by MPC. For the manipulator, only the point-to-point MPC approach is validated, as the arm operates in a constrained workspace with predictable obstacle-free regions.³

A. Global Planner: PRM, RRT and RRT*

The restaurant environment contains mostly static obstacles (tables, chairs, bar counters) that rarely change. To efficiently plan collision-free paths in configuration space, three sampling-based planners are evaluated:

Probabilistic Roadmap (PRM):

A multi-query planner that builds a reusable roadmap of the environment. PRM is particularly useful for repetitive tasks, such as navigating between the bar and various tables, as it reuses the same roadmap over multiple queries.

Rapidly-exploring Random Tree (RRT):

A single-query planner that incrementally grows a tree toward the goal. RRT is fast at finding feasible paths in complex spaces, particularly when handling non-holonomic constraints, but does not guarantee path optimality [6][7].

Rapidly-exploring Random Tree Star (RRT*):

An optimized extension of RRT that introduces a “rewiring” step during tree expansion, allowing convergence toward shorter paths and more optimal trajectories [3][7].

B. Local Controller: Model Predictive Control (MPC)

Model Predictive Control is used for both trajectory tracking (hierarchical approach) and direct point-to-point motion

²For configurations using a global planner, only trials where MPC does not fail due to convergence to local minima have been considered, in order to isolate the effect of the global planner.

³All planning algorithms were implemented from scratch. The CasADi library was used as the framework for the MPC formulation, while the PyBullet API was used for the implementation of sampling-based planners.

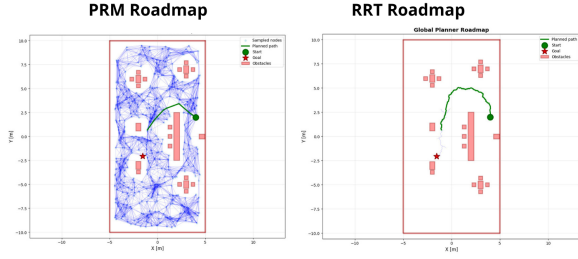


Fig. 3. Example of global roadmaps generated by PRM and RRT.

(standalone baseline). At each control step, the MPC predicts the robot's motion over a finite horizon and optimizes control inputs to minimize tracking error while satisfying kinematic, dynamic, and safety constraints [8][9]. Both controllers are implemented using CasADi [10] with the IPOPT nonlinear solver [11].

1) Implementation of MPC for the Base

The base MPC optimizes over the 3-DOF state space $\mathbf{x} = [x, y, \theta]^T$ with control inputs $\mathbf{u} = [v, \omega]^T$ (linear and angular velocities). The optimization problem over horizon N is formulated as:

$$\begin{aligned}
 \min_{\mathbf{x}_{0:N}, \mathbf{u}_{0:N-1}} \quad & \sum_{k=0}^{N-1} \left[w_p \|\mathbf{p}_k - \mathbf{p}_k^{\text{ref}}\|^2 + w_u \|\mathbf{u}_k\|^2 \right] + w_{\text{term}} \|\mathbf{p}_N - \mathbf{p}_N^{\text{ref}}\|^2 \\
 & + \sum_{i=1}^{M_{\text{static}}} \frac{w_{\text{obs}}}{\|\mathbf{p}_0 - \mathbf{p}_{\text{obs}}^i\|^2 - d_{\min}^2} + \sum_{j=1}^{M_{\text{dyn}}} \frac{w_{\text{dyn}} \cdot r_j}{d_{\text{pred}}^j + \epsilon} \\
 \text{s.t.} \quad & x_{k+1} = x_k + v_k \cos \theta_k \cdot \Delta t \\
 & y_{k+1} = y_k + v_k \sin \theta_k \cdot \Delta t \\
 & \theta_{k+1} = \theta_k + \omega_k \cdot \Delta t \\
 & \mathbf{x}_0 = \mathbf{x}_{\text{init}} \\
 & v_{\min} \leq v_k \leq v_{\max}, \quad \omega_{\min} \leq \omega_k \leq \omega_{\max} \\
 & x_{\min} \leq x_k \leq x_{\max}, \quad y_{\min} \leq y_k \leq y_{\max}
 \end{aligned} \tag{4}$$

where $\mathbf{p}_k = [x_k, y_k]^T$ is the 2D position, $\mathbf{p}_k^{\text{ref}}$ is the reference position⁴, and the dynamics follow Euler integration of the differential-drive kinematic model (Eq. 1). The state bounds $(x_{\min}, x_{\max})$ and $(y_{\min}, y_{\max})$ correspond to the physical room dimensions. Obstacle avoidance is implemented as soft constraints: static obstacles contribute a quadratic penalty when distance d_k^i violates the safety margin, while dynamic obstacles use a barrier function based on predicted distance $d_{k,\text{pred}}^j$ accounting for obstacle velocity.

2) Implementation of MPC for the Arm

The arm MPC operates in the 7-DOF joint space with state $\mathbf{s}_a = [\mathbf{q}_a^T, \dot{\mathbf{q}}_a^T]^T \in \mathbb{R}^{14}$ and control inputs $\ddot{\mathbf{q}}_a \in \mathbb{R}^7$

⁴In standalone MPC mode, $\mathbf{p}_k^{\text{ref}}$ is set to the fixed goal target for all k . In path following mode (hierarchical approach), $\mathbf{p}_k^{\text{ref}}$ is sampled from the global planner path at the corresponding lookahead distance.

(joint accelerations). The optimization problem is:

$$\begin{aligned}
 \min_{\mathbf{q}_{0:N}, \dot{\mathbf{q}}_{0:N-1}} \quad & \sum_{k=0}^{N-1} \left[w_q \|\mathbf{q}_k - \mathbf{q}_k^{\text{ref}}\|^2 + w_{\dot{q}} \|\dot{\mathbf{q}}_k\|^2 + w_{\ddot{q}} \|\ddot{\mathbf{q}}_k\|^2 \right] \\
 & + w_{\text{term}} \|\mathbf{q}_N - \mathbf{q}_N^{\text{ref}}\|^2 \\
 \text{s.t.} \quad & \mathbf{q}_{k+1} = \mathbf{q}_k + \dot{\mathbf{q}}_k \Delta t \\
 & \dot{\mathbf{q}}_{k+1} = \dot{\mathbf{q}}_k + \ddot{\mathbf{q}}_k \Delta t \\
 & \mathbf{q}_{\min} \leq \mathbf{q}_k \leq \mathbf{q}_{\max} \\
 & \dot{\mathbf{q}}_{\min} \leq \dot{\mathbf{q}}_k \leq \dot{\mathbf{q}}_{\max} \\
 & \boldsymbol{\tau}_{\min} \leq \boldsymbol{\tau}_k \leq \boldsymbol{\tau}_{\max}
 \end{aligned} \tag{5}$$

where $\mathbf{q}_k^{\text{ref}}$ is the target joint configuration, and the prediction model follows a double integrator. Joint torques $\boldsymbol{\tau}_k$ are computed via inverse dynamics using Pinocchio [12] and limited by actuator bounds. The cost penalizes joint error, velocity, and acceleration for smooth motion, with a terminal cost on the final configuration. The MPC operates in joint space, using IK to pre-compute target poses while enforcing end-effector orientation for grasping.

Advantages of MPC

- Predictive behavior for smooth, spill-free trajectories.
- Handles constraints: nonholonomic base, joint limits, obstacle avoidance.
- Allows integration of dynamic obstacles through online cost shaping.

MPC has been demonstrated to effectively stabilize high-DOF mobile manipulators under strict end-effector constraints [9], as it outputs safe velocity commands for the base and arm, ensuring collision avoidance and stable drink transport.

IV. RESULTS

This section presents the simulation setup and quantitative evaluation of the motion planning strategies. As the results are still pending, placeholder values are included in the tables.

A. Simulation Environment

The simulation is implemented in PyBullet using the gym_envs.urdf framework [13]. The environment represents a restaurant layout with a bar, several tables and chairs, and narrow passages between tables, creating a challenging navigation scenario.

The simulation is structured around four key components:

- **Task:** Pick up a drink from the bar, navigate the restaurant, and deliver it to a target table.
- **Robot Model:** Complete kinematic model of the Albert mobile manipulator, combining a Clearpath Boxer base with a 7-DOF Panda arm).
- **Obstacles:**
 - Static: tables, chairs, bar, walls.
 - Dynamic: moving blocks representing customers.
- **Payload:** A bottle that must be transported smoothly.

B. Quantitative Metrics – Base

In this section, we evaluate the performance of the three planning configurations we compare: standalone MPC, PRM combined with MPC, and RRT with MPC, using the metrics established in Section II-D.

TABLE I

SIMULATION METRICS FOR DIFFERENT PLANNING CONFIGURATIONS

Metric	MPC	+PRM	+RRT	+RRT*
Success Rate (%)	74	91	87	96
Planning Time (s)	0	97.22	1.85	24.87
Execution Time (s)	48.59	51.90	66.91	66.53
Global Planner Path Length (m)	-	7.02	8.80	7.37
Executed Distance (m)	6.49	7.11	9.40	7.5
Acceleration Norm (m/s^2)	0.94	0.79	1.08	0.99
Jerk Norm (m/s^3)	20.61	22.86	27.0	30.94

Table III summarizes the mean values collected across 30 experimental trials.⁵ This information allows us to compare the standalone local controller against the hierarchical configurations, and to quantify the impact of adding a global planner to the navigation task.

C. Quantitative Metrics – Arm

To evaluate whether a global planner is beneficial for the 7-DOF arm, we compare MPC-only against PRM+MPC and RRT+MPC using end-effector and control metrics.

TABLE II

ARM METRICS FOR MPC VS. PRM+MPC VS. RRT+MPC

Metric	MPC	+PRM	+RRT
Duration (s)	4.3500	4.1500	4.0500
EE Path Length (m)	1.2639	1.2551	1.2406
Final EE Error (m)	0.0166	0.0166	0.0172
Max Accel (rad/s^2)	17.1604	17.2615	17.4203
Planner Time (s)	-	6.4369	6.8092

As shown in Table II, adding PRM or RRT provides negligible improvements in duration, end-effector path length, and final error, while introducing a non-trivial planning overhead ($\approx 6-7$ s). Therefore, for this project the arm is controlled using MPC-only for all arm phases, and global planning is reserved for base navigation.

Fig. 4 shows the 3D end-effector trajectory during grasping. The arm first reaches a pre-grasp pose above the bottle and then descends in z to perform the grasp. Due to base placement and kinematic constraints, the approach in (x, y) follows a curved path rather than a straight line.

D. Comparison 1: Local vs. Hierarchical Planning

The biggest difference is in Success Rate: standalone MPC reaches 74%, while all hierarchical configurations exceed 85%. MPC alone struggles with local minima, whereas adding a global planner (PRM, RRT, or RRT*) gives a collision-free reference, allowing MPC to focus on trajectory smoothing rather than solving the full navigation problem.

⁵To ensure statistical significance, each configuration was tested over 30 different trials within the restaurant simulation.

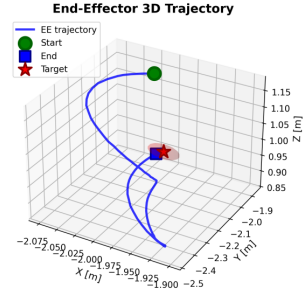


Fig. 4. 3D end-effector trajectory of the Panda arm using MPC. The grasping motion is divided into distinct phases: approaching the target (moving above the bottle), descending for grasping, lifting the drink, and holding it during transport.

As well, hierarchical planning introduces computational overhead: MPC does not spend time on planning, while PRM requires 97.22s and RRT* about 25s. Jerk Norms also increase from 20.61 m/s^3 (MPC) to 30.94 m/s^3 (+RRT*), indicating that following global paths with local adjustments can produce more aggressive control actions compared to the standalone smooth MPC.

E. Comparison 2: Global Planner Comparison

This subsection compares PRM, RRT, and RRT* combined with MPC. As shown in Table III, all three achieve high success rates (87–96%), showing that adding a global planner improves robustness over standalone MPC. In terms of planning time, PRM is the highest due to roadmap construction, while RRT* takes longer than RRT because it optimizes path quality. This trade-off is reflected in the global path length, where RRT* produces slightly shorter paths than RRT, while PRM yields the shortest paths overall.

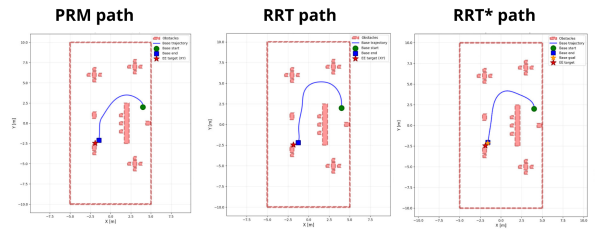


Fig. 5. Comparison of global paths generated by PRM, RRT, and RRT*.

Execution times and distances are similar across planners, suggesting that the MPC successfully smooths and adapts the global paths during execution. However, RRT and RRT* show higher acceleration and jerk, indicating less smooth trajectories compared to PRM.

Overall, PRM has lower execution times and provides smoother trajectories. While initial planning is slower due to roadmap construction, the roadmap can be reused in a largely static restaurant environment, enabling efficient and predictable drink delivery.

F. Comparison 3: Static vs. Dynamic Obstacles

This subsection evaluates MPC performance in environments with only static obstacles versus both static and dynamic obstacles.

TABLE III
SIMULATION METRICS FOR STATIC AND DYNAMIC OBSTACLE AVOIDANCE WITH MPC

Metric	Static Env.	Dynamic Env.
Success Rate (%)	74	36
Execution Time (s)	48.59	79.21
Total Executed Distance (m)	9.39	10.34
Acceleration Norm (m/s ²)	0.94	0.54
Jerk Norm (m/s ³)	20.61	16.72

As shown in the table, dynamic obstacles degrade performance: success rate drops as the MPC can get trapped in local minima or fail to reach the goal within the time limit. Execution time and distance increase due to avoidance maneuvers, while acceleration and jerk norms decrease slightly, reflecting more conservative motion to maintain safety.

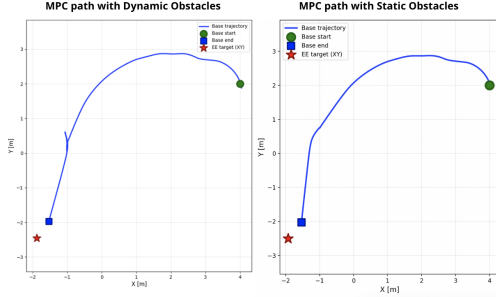


Fig. 6. Trajectories of the MPC controller: with dynamic obstacles (left) and in a static environment (right).

V. DISCUSSION AND CONCLUSION

A. Performance Analysis

The simulation results show that hierarchical planners significantly improve performance compared to standalone MPC. For example, the success rate increases from 74% for MPC alone to 91–96% when combined with PRM, RRT, or RRT*, demonstrating that providing a global reference path reduces the risk of local minima.

In terms of efficiency, metrics show that PRM has the highest planning time (97.22 s) due to roadmap construction, while RRT and RRT* plan faster, though RRT* also optimizes path quality. Execution times and total traveled distances are comparable across planners, suggesting MPC effectively smooths the global paths during execution.

Finally, PRM excels in repeated navigation tasks, such as serving drinks along similar paths in a restaurant, thanks to its reusable roadmap. RRT and RRT* would be preferable for one-time paths where fast planning is important. Overall, the hierarchical strategies make the robot’s navigation more robust, helping it avoid collisions and successfully complete tasks more often than using only the local planner.

B. Limitations

The proposed approach has several limitations:

- Sampling-based global planners (PRM/RRT) are not guaranteed to find optimal paths and may fail in very cluttered areas.
- RRT* requires longer planning times to converge to optimal solutions, which can limit responsiveness in highly dynamic scenarios.
- MPC performance depends on horizon length and tuning, and in dynamic environments it can be limited.

C. Future Work

Potential directions to extend this project include:

- Exploring alternative local planners, such as MPPI, for smoother trajectories.
- Evaluating other global planners, e.g., RRT-Connect, to improve path optimality.
- Improve the MPC implementation to reduce the likelihood of getting trapped in local minima.
- Implementing whole-body control for the high-dimensional mobile manipulator to improve coordination and timing between the arm and the base.

in future works I would add the wholebody control of the mobile manipulator in high dimensional for better communication base arm

These improvements could make the system more robust, efficient, and closer to real-world deployment in dynamic environments, such as a restaurant.

D. Conclusion

This project compared motion planning strategies for a mobile manipulator performing drink delivery in a restaurant, focusing on standalone MPC versus combining MPC with a global planner (PRM, RRT, or RRT*). The sequential planning strategy decomposes the task into manageable phases: navigation, grasping, transport, and delivery, while constraints make sure the robot moves smoothly and avoids collisions.

Simulation results demonstrate that hierarchical planning consistently outperforms standalone MPC, improving task success, robustness, and trajectory smoothness. As for the global planner, PRM outperforms for repeated navigation due to its reusable roadmap, while RRT and RRT* offer faster planning and optimized paths for single-query scenarios.

Overall, this study highlights the benefits of combining global and local planners for mobile manipulators, improving navigation, task success, and trajectory smoothness, and provides a foundation for future work in dynamic environments and real-world deployment.

Finally, this project can be accessed and tested through the GitHub repository [14]. In addition, recorded simulation videos of experiments without dynamic obstacle avoidance, another with the dyanmic obstacles, and the RRT* + MPC and MPC can be found on these references [15] [16] [17].

REFERENCES

- [1] T. Sandakalum and M. H. Ang Jr., "Motion Planning for Mobile Manipulators—A Systematic Review," *Machines*, vol. 10, no. 2, p. 97, 2022, doi: 10.3390/machines10020097.
- [2] N. Demir, H. Sucuoglu, I. Bogrekci and P. Demircioglu, "Structural & Dynamic Analyses and Simulation of Mobile Transportation Robot," *International Journal of 3D Printing Technologies and Digital Industry*, vol. 5, pp. 587-595, 2021, doi: 10.46519/ij3dptdi.949803.
- [3] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846-894, 2011, doi: 10.1177/0278364911406761.
- [4] T. Sandakalum and M. H. Ang, "Motion Planning for Mobile Manipulators—A Systematic Review," *Machines*, vol. 10, no. 2, article 97, 2022. [Online]. Available: <https://www.mdpi.com/2075-1702/10/2/97>
- [5] M. M. Ghazaei Ardakani, B. Olofsson, A. Robertsson, and R. Johansson, "Model Predictive Control for Real-Time Point-to-Point Trajectory Generation," *IEEE Transactions on Automation Science and Engineering*, vol. 16, no. 2, pp. 972-983, 2019.
- [6] T. Xu, "Recent advances in Rapidly-exploring random tree: A review," *Heliyon*, vol. 10, no. 11, p. e32451, 2024, doi: 10.1016/j.heliyon.2024.e32451.
- [7] Y. Wang, J. Tian, Z. Liu, H. Liu, J. Liu and L. Xie, "Path planning of manipulator based on improved RRT algorithm," *Journal of Physics: Conference Series*, vol. 2216, p. 012012, 2022, doi: 10.1088/1742-6596/2216/1/012012.
- [8] A. Mavrommati, C. Osorio, R. G. Valenti, A. Rajhans and P. J. Mosterman, "An Application of Model Predictive Control to Reactive Motion Planning of Robot Manipulators," 2021 IEEE 17th International Conference on Automation Science and Engineering (CASE), Lyon, France, 2021, pp. 915-920, doi: 10.1109/CASE49439.2021.9551432.
- [9] M. Osman, M. W. Mehrez, S. Yang, S. Jeon and W. Melek, "End-Effector Stabilization of a 10-DOF Mobile Manipulator using Non-linear Model Predictive Control," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 9772-9777, 2020, doi: 10.1016/j.ifacol.2020.12.2658.
- [10] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings and M. Diehl, "CasADi: A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1-36, 2019, doi: 10.1007/s12532-018-0139-4.
- [11] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25-57, 2006, doi: 10.1007/s10107-004-0559-y.
- [12] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiriaux, O. Stasse and N. Mansard, "The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," in *Proc. IEEE Int. Symp. on System Integration (SII)*, 2019, pp. 614-619, doi: 10.1109/SII.2019.8700380.
- [13] Spahn, M., *gym_envs_urdf*, GitHub repository. Available: https://github.com/maxspahn/gym_envs_urdf.
- [14] *Planning and Decision Making Project*, GitHub repository, 2024. Available: <https://github.com/Simoneutili/PlanningandDecisionMakingProject/tree/master>.
- [15] *MPC without Dynamic Obstacle Avoidance – Simulation Test*, YouTube video, 2024. Available: <https://youtu.be/gMZc4qK0IB0>.
- [16] *MPC with Dynamic Obstacle Avoidance – Simulation Test*, YouTube video, 2024. Available: https://youtu.be/tc6glKit_Fk.
- [17] *RRT* + MPC Motion Planning – Simulation Test*, YouTube video, 2024. Available: <https://www.youtube.com/watch?v=8MRPteQAve4>.